Data Analysis Programming
Semester 2026-I
Guide: Airflow and PostgreSQL ETL Pipeline

`Eng. Carlos Andrés Sierra, M.Sc.`
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Introduction

This guide walks you, step by step, through building a small but complete **Extract, Transform, Load** (ETL) pipeline. You will run Apache Airflow and PostgreSQL in Docker containers, ingest earthquake events from a public REST API, curate them into a relational table, and compute summary statistics that are saved as Parquet files.

The pipeline follows a **medallion architecture** with three layers:

- **Bronze**: raw JSON files, exactly as the API returns them.
- **Silver**: cleaned and validated records stored in PostgreSQL.
- **Gold**: aggregated summaries stored as Parquet files.

Each layer is produced by an Airflow DAG (a Directed Acyclic Graph, that is, a workflow of tasks with a defined order and schedule). The three DAGs are:

1. `earthquake_ingestion`: runs every minute and saves raw API responses to the bronze layer.
2. `earthquake_curation`: runs every five minutes, cleans the bronze files, and loads them into PostgreSQL.
3. `earthquake_analysis`: runs after curation finishes, calculates statistics from PostgreSQL, and writes Parquet files to the gold layer.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Learning Outcomes

After completing this guide, you should be able to:

- Describe the role of Airflow's scheduler, webserver, and metadata database.
- Start a multi-container environment with Docker Compose and verify its health.
- Write Airflow DAGs that call an external API, read and write files, and use a database connection.
- Make a pipeline idempotent, so that re-running a task does not duplicate data.
- Diagnose the most common Airflow and Docker errors.

Prerequisites

- Docker Engine and Docker Compose v2 (`docker compose version` must work).
- A terminal (Linux, macOS, or WSL 2 on Windows) with `curl`.
- Basic Python (functions, dictionaries, and reading JSON).
- A PostgreSQL client such as DBVisualizer, or `psql`, to inspect the database.
- About 4 GB of free RAM and 5 GB of free disk space.

Project Layout

Create the following folder structure. Every file in this guide is placed inside it, and the paths in the rest of the guide are relative to the project root (`earthquake-etl/`).

```
earthquake-etl/
  docker-compose.yml
  .env
  init-db/
    01-create-etl-database.sql
  airflow/
    dags/
      earthquake_ingestion.py
      earthquake_curation.py
      earthquake_analysis.py
  datalake_bronze/
  datalake_silver/
  datalake_gold/
```

Run the following commands from a terminal to create the folders:

```
mkdir -p
    earthquake-etl/{init-db,airflow/dags,datalake_bronze,datalake_silver,datalake_gold}
cd earthquake-etl
```

Step 1: Configure the Container Environment

1.1 Set the Airflow user ID

On Linux, the containers must write to the `datalake_*` folders with your user ID. Create the `.env` file in the project root:

```
echo "AIRFLOW_UID=$(id -u)" > .env
```

On macOS and Windows (WSL 2) you can keep this file as it is. Its content should look like this:

```
AIRFLOW_UID=1000
```

1.2 Create the database initialization script

PostgreSQL runs every `.sql` file in `init-db/` the first time its data volume is created. This script creates a separate database and user for the ETL data, so that Airflow metadata and business data are kept apart. Create `init-db/01-create-etl-database.sql`:

```
CREATE USER etl_ud WITH PASSWORD 'P4$w0rd';
CREATE DATABASE earthquakes_ud OWNER etl_ud;
```

1.3 Write the Docker Compose file

Create `docker-compose.yml` in the project root. The file defines five services: one PostgreSQL server, a one-time initialization job for Airflow, the webserver, the scheduler, and the shared settings that the Airflow services reuse.

```
x-airflow-common: &airflow-common
  image: apache/airflow:2.10.4
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
    postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW_CONN_POSTGRES_ETL: postgresql://etl_ud:P4$$w0rd@postgres:5432/earthquakes_ud
    _PIP_ADDITIONAL_REQUIREMENTS: pandas polars pyarrow
  volumes:
    - ./airflow/dags:/opt/airflow/dags
    - ./datalake_bronze:/opt/airflow/datalake_bronze
    - ./datalake_gold:/opt/airflow/datalake_gold
  user: "${AIRFLOW_UID:-50000}:0"
  depends_on: &airflow-depends
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
```

```
      POSTGRES_DB: airflow
    ports:
      - "5440:5432"  # host port 5440: ports 5432 and 5433 may already be used by a local
    PostgreSQL
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./init-db:/docker-entrypoint-initdb.d
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U airflow"]
      interval: 5s
      retries: 10

  airflow-init:
    <<: *airflow-common
    command:
      - bash
      - -c
      - |
        airflow db migrate &&
        airflow users create \
          --username airflow --password airflow \
          --firstname Admin --lastname User \
          --role Admin --email admin@example.com
    depends_on:
      <<: *airflow-depends
    restart: "no"

  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports:
      - "8080:8080"
    depends_on:
      airflow-init:
        condition: service_completed_successfully

  airflow-scheduler:
    <<: *airflow-common
    command: scheduler
    depends_on:
      airflow-init:
        condition: service_completed_successfully

volumes:
  postgres-data:
```

Why these settings? The `LocalExecutor` runs tasks as processes on the scheduler container, which is enough for this guide. A production system would use the `CeleryExecutor` or `KubernetesExecutor` with separate worker containers. The `AIRFLOW_CONN_POSTGRES_ETL` variable creates an Airflow connection named `postgres_etl` automatically, so you do not have to create it in the user interface. The `_PIP_ADDITIONAL_REQUIREMENTS` variable installs the Python libraries used by the DAGs when the containers start. This is convenient for learning, but a production image should install dependencies at build time.

Step 2: Start the Services

2.1 Launch the stack

From the project root, run:

```
docker compose up -d
```

The first start downloads the images and installs the Python packages, so it can take several minutes. Check the progress with:

```
docker compose ps
docker compose logs -f airflow-init
```

The `airflow-init` container should finish with an exit code of 0 and a status of `Exited (0)`. The `postgres` container should show `healthy`.

2.2 Open the Airflow user interface

Open `http://localhost:8080` in a browser and sign in with user `airflow` and password `airflow`. The DAG list is empty for now, because the DAG files are added in Step 4.

2.3 Connect to PostgreSQL

In DBVisualizer (or `psql`), create a connection with the following settings:

Field	Value
Host	`localhost`
Port	`5440`
Database	`earthquakes_ud`
User	`etl_ud`
Password	`P4$w0rd`

Run `SELECT current_user;` to confirm the connection. The result should be `etl_ud`.

2.4 Verify the Airflow connection

The connection was created from the `AIRFLOW_CONN_POSTGRES_ETL` environment variable. Airflow reads such connections directly from the environment and does not store them in its metadata database, so the **Admin** → **Connections** page in the user interface stays empty. This is expected and does not mean the connection is missing.

To check the connection, run the following command from the project root:

```
docker compose exec airflow-webserver airflow connections get postgres_etl
```

The output should show the type `postgres`, the host `postgres`, the database `earthquakes_ud`, and the login `etl_ud`. The password is printed in clear text in this output, so do not paste it into shared logs or screenshots.

Step 3: Build the Bronze Layer (DAG 1)

Objective

Fetch earthquake events from the USGS Earthquake Hazards Program every minute and save each response as a JSON file in the `datalake_bronze/` folder. The API documentation is available at `https://earthquake.usgs.gov/fdsnws/event/1/`.

3.1 Create the DAG file

Create `airflow/dags/earthquake_ingestion.py` with the following content:

```
import json
from datetime import datetime, timedelta
from pathlib import Path

import requests
from airflow import DAG
from airflow.operators.python import PythonOperator

USGS_URL = "https://earthquake.usgs.gov/fdsnws/event/1/query"
BRONZE_DIR = Path("/opt/airflow/datalake_bronze")


def extract_events(data_interval_start, data_interval_end, **_):
    """Call the USGS API for the time window of this run."""
    params = {
        "format": "geojson",
        "starttime": data_interval_start.strftime("%Y-%m-%dT%H:%M:%S"),
        "endtime": data_interval_end.strftime("%Y-%m-%dT%H:%M:%S"),
    }
    response = requests.get(USGS_URL, params=params, timeout=30)
    response.raise_for_status()
    return response.json()


def save_to_bronze(data_interval_start, ti, **_):
    """Write the raw API response to a timestamped JSON file."""
    payload = ti.xcom_pull(task_ids="extract")
    BRONZE_DIR.mkdir(parents=True, exist_ok=True)
    stamp = data_interval_start.strftime("%Y%m%d_%H%M%S")
    path = BRONZE_DIR / f"earthquakes_{stamp}.json"
    with path.open("w", encoding="utf-8") as file:
        json.dump(payload, file)
    return str(path)


with DAG(
    dag_id="earthquake_ingestion",
    description="Fetch USGS earthquakes every minute into the bronze layer",
    schedule="* * * * *",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    default_args={"retries": 1, "retry_delay": timedelta(seconds=30)},
    tags=["bronze", "usgs"],
```

```
) as dag:
    extract = PythonOperator(task_id="extract", python_callable=extract_events)
    save = PythonOperator(task_id="save_to_bronze", python_callable=save_to_bronze)

    extract >> save
```

3.2 Understand the code

- **Schedule**: `"* * * * *"` is a cron expression meaning "every minute".
- **Data interval**: Airflow passes `data_interval_start` and `data_interval_end` to each run. Using them, instead of the current time, makes every run deterministic: re-running the run for 12:03 always requests the same time window.
- **XCom**: `extract` returns the JSON payload, and Airflow stores it in its metadata database so that `save_to_bronze` can read it with `ti.xcom_pull`. XCom is intended for small values. For large payloads, write to a file in the first task and pass only the file path.
- **Timestamp naming**: the file name uses the start of the interval, for example `earthquakes_20260315_142000.json`. Curation in Step 4 relies on this pattern.
- **Times are in UTC**: Airflow and the USGS API both use UTC. Keep this in mind when you read timestamps.

3.3 Enable and test the DAG

1. In the Airflow interface, find `earthquake_ingestion` and switch it on with the toggle on the left. Airflow pauses new DAGs by default, which is why this step is needed.
2. Click the DAG name, then **Trigger DAG** to run it once manually.
3. Open the run and check that both tasks are green (`success`).
4. On your computer, confirm that a file appeared:

```
ls -lh datalake_bronze/
```

Open the file and confirm that it contains a top-level `features` list. Quiet periods may produce a file with an empty list, which is valid: the pipeline must handle empty windows.

Step 4: Build the Silver Layer (DAG 2)

Objective

Every five minutes, read the bronze files that belong to the five-minute window, clean and validate the events, and load them into the `earthquakes_curated` table in PostgreSQL. Running the DAG twice for the same window must not create duplicate rows.

4.1 Create the DAG file

Create `airflow/dags/earthquake_curation.py` with the following content:

```
import json
import re
from datetime import datetime
from pathlib import Path

import pandas as pd
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook

BRONZE_DIR = Path("/opt/airflow/datalake_bronze")
CONN_ID = "postgres_etl"
FILE_PATTERN = re.compile(r"earthquakes_(\d{8}_\d{6})\.json$")
COLUMNS = [
    "event_id", "event_time", "magnitude", "place",
    "latitude", "longitude", "depth_km", "event_type",
]

DDL = """
CREATE TABLE IF NOT EXISTS earthquakes_curated (
    event_id    TEXT PRIMARY KEY,
    event_time  TIMESTAMPTZ NOT NULL,
    magnitude   DOUBLE PRECISION NOT NULL,
    place       TEXT,
    latitude    DOUBLE PRECISION,
    longitude   DOUBLE PRECISION,
    depth_km    DOUBLE PRECISION,
    event_type  TEXT
);
"""


def create_table():
    """Create the curated table if it does not exist yet (safe to run repeatedly)."""
    PostgresHook(postgres_conn_id=CONN_ID).run(DDL)


def read_bronze_window(window_start, window_end):
    """Return the flattened features from bronze files stamped inside [start, end)."""
    records = []
    for path in sorted(BRONZE_DIR.glob("earthquakes_*.json")):
        match = FILE_PATTERN.search(path.name)
        if not match:
            continue
        file_time = datetime.strptime(match.group(1), "%Y%m%d_%H%M%S")
        if not window_start <= file_time < window_end:
            continue
        with path.open(encoding="utf-8") as file:
            payload = json.load(file)
        for feature in payload.get("features", []):
            props = feature.get("properties") or {}
            lon, lat, depth = (feature.get("geometry") or {}).get(
```

```
                "coordinates", [None, None, None]
            )
            records.append({
                "event_id": feature.get("id"),
                "event_time": props.get("time"),  # epoch milliseconds
                "magnitude": props.get("mag"),
                "place": props.get("place"),
                "latitude": lat,
                "longitude": lon,
                "depth_km": depth,
                "event_type": props.get("type"),
            })
    return records


def curate_and_load(data_interval_start, data_interval_end, **_):
    """Clean the window's records and upsert them into PostgreSQL."""
    window_start = data_interval_start.replace(tzinfo=None)
    window_end = data_interval_end.replace(tzinfo=None)
    df = pd.DataFrame(read_bronze_window(window_start, window_end))

    if df.empty:
        print("No events in this window; nothing to load.")
        return 0

    # Validation: drop rows without an identifier, magnitude, or time.
    df = df.dropna(subset=["event_id", "magnitude", "event_time"])
    # Deduplication: the same event can appear in overlapping API responses.
    df = df.drop_duplicates(subset="event_id", keep="last")
    # Standardization: timestamps as UTC datetimes, text trimmed, missing places labeled.
    df["event_time"] = pd.to_datetime(df["event_time"], unit="ms", utc=True)
    df["place"] = df["place"].fillna("unknown").str.strip()

    rows = [
        tuple(
            None if pd.isna(value)
            else value.to_pydatetime() if isinstance(value, pd.Timestamp)
            else value
            for value in row
        )
        for row in df[COLUMNS].itertuples(index=False, name=None)
    ]

    # Idempotent load: rows with an existing event_id are updated, not duplicated.
    PostgresHook(postgres_conn_id=CONN_ID).insert_rows(
        table="earthquakes_curated",
        rows=rows,
        target_fields=COLUMNS,
        replace=True,
        replace_index="event_id",
    )
    print(f"Loaded {len(rows)} events.")
    return len(rows)
```

```
with DAG(
    dag_id="earthquake_curation",
    description="Curate bronze earthquake files and load them into PostgreSQL",
    schedule="*/5 * * * *",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    default_args={"retries": 1},
    tags=["silver", "postgres"],
) as dag:
    prepare = PythonOperator(task_id="create_table", python_callable=create_table)
    curate = PythonOperator(task_id="curate_and_load", python_callable=curate_and_load)

    prepare >> curate
```

4.2 Understand the code

- **Window filter**: a run that covers 12:00–12:05 reads only files stamped from 12:00:00 up to, but not including, 12:05:00. Consecutive windows never overlap, so no file is read twice by different runs.
- **Validation and cleaning**: records with missing keys are discarded and the rest are standardized. In a real project you would record the rejected rows, which is a good practice for data-quality reporting.
- **Idempotency**: `insert_rows` with `replace=True` generates an `INSERT ... ON CONFLICT (event_id) DO UPDATE` statement. Re-running a window updates existing events instead of inserting duplicates.
- **Pandas missing values**: `NaN` is converted to `None`, because PostgreSQL drivers store `NULL` and not `NaN`.

4.3 Enable and test the DAG

1. Switch on `earthquake_curation` in the Airflow interface.
2. Trigger one run manually. Note that the manual run uses the same five-minute window logic, so it may load zero events if no bronze file falls inside that window.
3. Wait for the scheduled runs. After a few minutes, query the table in DBVisualizer:

```
SELECT COUNT(*) AS events, MAX(event_time) AS latest_event
FROM earthquakes_curated;
```

Run the same query twice. The count must not grow when no new bronze files exist, which demonstrates that the load is idempotent.

Step 5: Build the Gold Layer (DAG 3)

Objective

After the curation DAG finishes its run for a window, calculate summary statistics for the last 24 hours from PostgreSQL and save them as a Parquet file in `datalake_gold/`.

5.1 Create the DAG file

Create `airflow/dags/earthquake_analysis.py` with the following content:

```
from datetime import datetime
from pathlib import Path

import polars as pl
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.sensors.external_task import ExternalTaskSensor

GOLD_DIR = Path("/opt/airflow/datalake_gold")
CONN_ID = "postgres_etl"
QUERY = """
SELECT event_id, event_time, magnitude, depth_km
FROM earthquakes_curated
WHERE event_time >= NOW() - INTERVAL '24 hours';
"""


def compute_statistics(data_interval_end, **_):
    """Aggregate the last 24 hours of events and write one Parquet file."""
    pandas_df = PostgresHook(postgres_conn_id=CONN_ID).get_pandas_df(QUERY)
    events = pl.from_pandas(pandas_df)

    stats = events.select(
        pl.len().alias("event_count"),
        pl.col("magnitude").mean().alias("avg_magnitude"),
        pl.col("magnitude").max().alias("max_magnitude"),
        pl.col("depth_km").mean().alias("avg_depth_km"),
    ).with_columns(
        pl.lit(data_interval_end.replace(tzinfo=None)).alias("snapshot_end"),
    )

    GOLD_DIR.mkdir(parents=True, exist_ok=True)
    stamp = data_interval_end.strftime("%Y%m%d_%H%M%S")
    path = GOLD_DIR / f"earthquake_stats_{stamp}.parquet"
    stats.write_parquet(path)
    print(stats)
    return str(path)


with DAG(
    dag_id="earthquake_analysis",
    description="Compute 24-hour earthquake statistics and save them as Parquet",
    schedule="*/5 * * * *",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    default_args={"retries": 1},
    tags=["gold", "analysis"],
) as dag:
    wait_for_curation = ExternalTaskSensor(
        task_id="wait_for_curation",
        external_dag_id="earthquake_curation",
```

```
        external_task_id="curate_and_load",
        mode="reschedule",
        timeout=600,
    )
    compute = PythonOperator(task_id="compute_statistics",
    python_callable=compute_statistics)

    wait_for_curation >> compute
```

5.2 Understand the code

- **ExternalTaskSensor**: this task waits until the curation task with the same logical date has succeeded. `mode="reschedule"` releases the worker slot while it waits.
- **Read from PostgreSQL**: `PostgresHook.get_pandas_df` runs the query and returns a pandas DataFrame, which is converted to a Polars DataFrame for the aggregation.
- **Parquet output**: Parquet is a columnar, compressed format. It is smaller and faster to read than CSV, and it keeps column types.

5.3 Enable and test the DAG

1. Switch on `earthquake_analysis`.
2. Confirm in the Graph view that `wait_for_curation` turns green before `compute_statistics` runs.
3. Check the gold layer:

```
import polars as pl
print(pl.read_parquet("datalake_gold/earthquake_stats_20260315_142000.parquet"))
```

Replace the file name with one that exists in your `datalake_gold/` folder.

Step 6: Verify the Complete Pipeline

Use the following checklist. Every item must pass before you move on to the next workshop.

1. All three DAGs are switched on and show recent successful runs in the Airflow interface.
2. `datalake_bronze/` contains JSON files named `earthquakes_YYYYMMDD_HHMMSS.json`.
3. The table `earthquakes_curated` exists in the `earthquakes_ud` database and has a primary key on `event_id`.
4. Re-running `earthquake_curation` for the same window does not change the row count.
5. `datalake_gold/` contains at least one `earthquake_stats_*.parquet` file.

Troubleshooting

Symptom	Likely cause and fix
`PermissionError` on `datalake_*` folders	`AIRFLOW_UID` in `.env` does not match your user. Set it again, run `docker compose down`, then `docker compose up -d`.
`airflow-init` exits with an error	Read its logs with `docker compose logs airflow-init`. A failed database migration usually means the `postgres` service was not healthy yet.
`connection refused` to `postgres`	Check `docker compose ps`. The service must show `healthy`.
`address already in use` on port `5440` or `5432`	Another program, such as a local PostgreSQL server, already uses that host port. Change the left-hand number of the `ports` entry in `docker-compose.yml` to a free port, then run `docker compose up -d` again.
`database "earthquakes_ud" does not exist`	The init script runs only when the volume is created. Run `docker compose down -v` to delete the volume, then start again. This removes all data.
DAG does not appear in the list	Open the DAG import errors banner in the interface, or run `docker compose exec airflow-scheduler airflow dags list-import-errors`.
USGS request fails with HTTP 400	Check the time format `YYYY-MM-DDTHH:MM:SS` and that `starttime` is earlier than `endtime`.
Analysis task waits forever	The curation run for that logical date failed or was not scheduled. Check `earthquake_curation` first.

Stopping and Cleaning Up

To stop the containers and keep the data, run:

```
docker compose down
```

To stop the containers and delete the PostgreSQL volume as well, run `docker compose down -v`. Use this command only when you want to start again from an empty database.

References

Apache Software Foundation. (2025). *Apache Airflow documentation* (version 2.10). https://airflow.apache.org/docs/apache-airflow/2.10.4/

Docker Inc. (2025). *Docker Compose documentation*. https://docs.docker.com/compose/

PostgreSQL Global Development Group. (2025). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

U.S. Geological Survey. (2025). *Earthquake catalog API: FDSN event web service*. https://earthquake.usgs.gov/fdsnws/event/1/